TP4 : générateur de changelog et charte d'usage

Durée : 60 min **Partie** : P4 (combiner les outils et vérifier le résultat)

Objectif

Construire un générateur de CHANGELOG testé, créer son fichier de prompt réutilisable et rédiger une charte d'usage. Répartir les tâches entre ChatGPT et Copilot, puis justifier les décisions prises.

Prérequis

- Démo 5 et TP2 ; environnement préparé avec `uv sync`.
- Ouvrir `tps/tp4_projet_complet/` seul dans VS Code. Choisir la cible **Local** pour les fichiers `.prompt.md`.
- Fichiers fournis : `besoin.md`, `commits.txt`, `changelog.py`, `test_acceptation_tp4.py`, `charte_ia_modele.md`.
- Conserver les signatures de `changelog.py` et les tests d'acceptation. Le jeu `commits.txt` suffit, sans historique Git à créer.

Partie 1 — Cadrer avec ChatGPT (7 min)

Donner `besoin.md` et les docstrings de `changelog.py`. Demander une spécification d'une page maximum : règles, critères vérifiables, cas limites et hors périmètre. Répondre aux questions, relire, puis enregistrer `spec.md`. Vérifier une règle contre une docstring et un test.

Partie 2 — Écrire les règles du projet (5 min)

Faire créer un `AGENTS.md` court. Utiliser ces lignes comme point de départ, puis faire préciser les conventions propres au projet :

```
## Projet
Générateur de changelog Python. Référence : spec.md.
## Commandes
uv run pytest -q
uv run python changelog.py commits.txt 2.0.0 --date 2026-10-09
## Tests
Ne pas modifier test_acceptation_tp4.py ni les signatures fournies.
Les résultats attendus viennent des règles, pas du code existant.
```

Vérifier chaque commande avant de garder le fichier.

Partie 3 — Planifier, implémenter et tester (23 min)

Demander un plan à Copilot pour implémenter `spec.md` dans `changelog.py`, avec des tests dans `test_changelog.py`. Relire le plan, puis faire appliquer les étapes. Relire les diffs et lancer les tests après les changements.

Si l'analyse d'une ligne est délicate, donner cet exemple de découpage à l'assistant :

```
ligne = "9f2c1ab|2026-10-08|feat(api)!: supprime la route /v1"
champs = ligne.strip().split("|", 2)
# Vérifier les trois champs, puis analyser le message selon la docstring.
```

Lancer `uv run python changelog.py commits.txt 2.0.0 --date 2026-10-09`. Contrôler le changement incompatible `9f2c1ab` et la ligne de fusion rejetée `4a7d6bc`. Conserver un exemple de règle précisée, de plan ajusté ou de proposition corrigée.

Partie 4 — Rendre la tâche réutilisable (10 min)

Créer `.github/prompts/changelog.prompt.md` avec **Chat: New Prompt File**, puis le lancer avec `/changelog` dans la cible **Local**. Ce point de départ peut aider :

```
---
name: changelog
agent: agent
---
Prépare la version ${input:version:2.1.0} depuis commits.txt.
Lance uv run python changelog.py commits.txt ${input:version:2.1.0}.
Montre les lignes ignorées et la section produite avant insertion.
Insère sous le titre de CHANGELOG.md, en conservant les versions précédentes.
Si la version existe déjà, signale-le sans créer de doublon.
```

Relire le fichier de prompt, vérifier la section avant insertion, puis relancer pour la même version et vérifier qu'elle n'est pas dupliquée.

Partie 5 — Rédiger la charte (10 min)

En binôme, remplir `charte_ia_modele.md` et enregistrer `charte_ia.md`. Rédiger trois règles : outil pour une tâche précise, données à exclure d'un prompt, vérification avant de garder du code. Donner un exemple vécu pour chaque règle. Préparer la restitution de deux minutes du bilan collectif.

Partie 6 — Vérifier et expliquer (5 min)

Lancer `uv run pytest -q`. Expliquer une décision prise pendant le projet et la vérification qui la justifie. Montrer le changelog, le fichier de prompt et la charte.

Critères de réussite

- Les 13 tests fournis et vos tests passent, sans modification des signatures ni des tests d'acceptation.
- `spec.md`, `AGENTS.md` et `.github/prompts/changelog.prompt.md` sont relus et utilisables.
- La commande produit le changelog attendu ; `/changelog` ne duplique pas une version.
- `charte_ia.md` contient trois règles avec leurs exemples.

Corrigé

Le dossier `corrige/` contient le script, les tests, la spécification, les règles du projet, le fichier de prompt et un exemple de charte. `corrige/README.md` décrit les vérifications.